

MoonCollections 开发报告

2026 MoonBit 国产开源生态竞赛（个人赛）

一、项目概述

MoonCollections 是为 2026 MoonBit 国产开源生态竞赛开发的纯 MoonBit 数据结构库。项目目标是为 MoonBit 生态补充标准库中缺失的常用数据结构，包括保序映射、保序集合、位图、布隆过滤器、LRU 缓存、并查集和频率计数器，共 7 个模块。开发周期：2026 年 6 月 | 代码规模：源码 1,497 行 / 测试 1,536 行 / 合计 3,033 行 | 测试覆盖：158 个单元测试全部通过 | 依赖：零第三方依赖，仅使用 MoonBit 标准库 | 许可证：MIT

二、技术架构

2.1 模块划分

项目按模块化组织，每个模块为独立子包：bitmask (BitSet 位图)、indexmap (IndexMap 保序映射)、orderedset (OrderedSet 保序集合，依赖 IndexMap)、bloomfilter (BloomFilter 概率集合)、lrucache (LRUCache 缓存，依赖 IndexMap)、unionfind (UnionFind 并查集)、countmap (CountMap 计数器)。

2.2 设计原则

- 纯 MoonBit 实现：所有模块均为原生代码，不使用 FFI，确保 Native/Wasm/JS 跨平台
- 渐进复杂度：从简单结构 (UnionFind、BitSet) 到组合结构 (IndexMap -> OrderedSet -> LRUCache)
- 类型安全：充分利用 trait 系统提供编译期约束 (Hash + Eq) 和调试支持 (Show)
- API 一致性：统一命名惯例，统一迭代器模式，统一构造器模式 new() / from_array()

2.3 依赖关系

BitSet / UnionFind 为独立模块，仅依赖 Array 和基础类型。CountMap 依赖 @hashmap.HashMap。IndexMap 依赖 @hashmap.HashMap + Array 实现双存储架构。OrderedSet 和 LRUCache 委托给 IndexMap，复用其保序逻辑。BloomFilter 独立，使用 Array[Byte] + @math 包。

三、各模块实现细节

3.1 BitSet (312行)

以 Array[Byte] 为存储，每字节 8 位。选择固定大小：集合运算需要双方位数匹配，resize() 提供显式变换。count() 使用硬件 popcnt，iter() 按字节惰性遍历。集合运算 (union/intersect/difference/symmetric_diff) 均创建新实例返回，保持不可变风格。关系运算 (is_subset/superset/disjoint) 利用位与运算批量判断。

3.2 IndexMap (404行)

双存储：Array[K] 维护插入顺序 + HashMap[K,(Int,V)] 维护键到(索引,值)映射。set 时已存在键只更新值不变位置。remove 提供两种策略：shift_remove O(n) 保序 / swap_remove O(1) 不保序，沿用 Rust indexmap 的设计。retain 原地过滤并同步索引，sort_keys 冒泡排序 + 索引更新。这是整个库中最复杂的模块。

3.3 OrderedSet (225行)

完全委托给 IndexMap[K,Unit]，将集合语义映射为映射语义。代码量减少 60% 以上，且 IndexMap 的优化和修复自动受益。实现了标准库 HashSet 所没有的集合运算（并集/交集/差集/对称差）和子集判定。

3.4 BloomFilter (138行)

利用 MoonBit 内置 Hash trait 生成哈希值，通过种子组合派生多个哈希函数。位数组大小和哈希数按标准公式自动优化： $m = -n \cdot \ln(p) / \ln^2(2)$ ， $k = (m/n) \cdot \ln(2)$ 。不实现删除（标准布隆过滤器不支持，会导致假阴性）。

3.5 LRUCache (125行)

委托给 IndexMap，利用其保序特性实现 LRU 驱逐。get() 通过 remove + reinsert 将元素移到末尾（最近使用），put() 超容量时移除首元素（最久未使用）。peek() 不改变顺序。set_capacity 支持动态缩容。

3.6 UnionFind (104行)

双 Array 存储 parent 和 rank。find 实现路径压缩 (parent[x] = parent[parent[x]])，union 实现按秩合并。均摊复杂度接近 $O(\alpha(n))$ ，实际可视为常数时间。

3.7 CountMap (156行)

委托给 HashMap[K,Int]。decrement 到 0 时自动 remove 清理。most_common_n 使用冒泡排序 ($O(n^2)$)，因为标准库无内建排序且 CountMap 的键数通常不大。

四、测试策略

每个模块有独立 *_test.mbt 文件，使用 MoonBit 内置 test 框架。测试原则：每 API

模块	测试数	覆盖重点
BitSet	27	位操作、集合运算、子集关系、迭代、越界安全
IndexMap	26	增删改查、顺序保持、retain/sort/pop、迭代器、Eq
OrderedSet	18	集合运算、子集关系、去重、自反操作
BloomFilter	12	添加/查询、零误判保证、大批量、类型兼容
LRUCache	17	LRU驱逐、peek、容量变更、lru/mru、迭代顺序
UnionFind	12	合并/连通、路径压缩、集合计数、reset
CountMap	15	增减、topN、merge、from_array、清空
合计	158	

五、遇到的问题与解决方案

- MoonBit 语法适应：C-style 的 `~` 运算符不可用，改用 `.lnot()` 方法。位运算运算符 `^` / `|` / `&` 可用，但按位取反必须用方法调用。
- 迭代器模式：MoonBit 使用 `Iter::new(fn) + return Some + nobreak { None }` 的闭包模式，与常见语言的 `yield` 语法不同。通过研究标准库 `HashMap` 的 `iter()` 实现理解。
- 结构体字段可变性：`mut` 标记的是字段可否整体替换而非内部可否修改。通过索引修改 `Array` 不需 `mut`，但 `clear()` 中整体替换字段 (`self.map = new HashMap`) 需要。
- Map 字面量歧义：`None => {}` 被解析为 `Map` 字面量，应使用 `None => ()` 表示 `Unit` 值。
- HashMap 包引用：子包需在 `moon.pkg` 中 `import moonbitlang/core/hashmap`，代码中通过 `@hashmap.HashMap` 引用。
- 浮点运算：`p.ln()` 报错，应使用 `@math.ln(p)` 的函数式调用。
- Range 运算符优先级：`0..<len - 1` 解析为 `(0..<len) - 1`，需要写成 `0..<(len - 1)`。

六、工程实践

CI/CD: GitHub Actions 配置 `moon check / build / test` 自动流程。Git 工作流: 16 次提交，每次对应独立逻辑单元（实现-测试-文档递进）。双仓库维护：GitHub + GitLink 同步推送，HTTPS 认证。

七、未来展望

- 性能优化: `IndexMap` 可通过墓碑标记延迟清理优化 `shift-remove`; `CountMap` 可用堆排序替换冒泡排序
- 功能扩展: `Trie` 前缀树、`PriorityQueue` 扩展、序列化支持、Counting `BloomFilter`
- 生态贡献: 发布到 `mooncakes.io` 包注册表，编写使用教程，根据社区反馈持续改进

八、总结

MoonCollections 项目完成了 7 个数据结构模块的完整实现、测试和文档。遵循纯 MoonBit、零依赖、类型安全的设计原则，为 MoonBit 生态贡献了一套实用且可扩展的基础数据结构库。所有代码通过编译和测试验证，双仓库部署就绪，CI 配置完成，具备作为竞赛提交项目的完整性和工程质量。